

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Отчет по учебной практике

«Навигатор по смыслу» -сравнение моделей семантического поиска

Выполнил студент группы БВТ2503

Герасимчук Виктория Сергеевна

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Постановка задачи	3
Используемые данные	4
Выбор моделей эмбедингов	5
ХОД ВЫПОЛНЕНИЯ РАБОТЫ	6
1. Подготовка среды разработки	6
2. Загрузка и анализ данных	6
3. Загрузка эмбединг-моделей	7
4. Разработка функции оценки модели	7
5. Оценка моделей и сбор результатов	7
6. Определение лучшей модели	8
7. Визуализация эмбедингов с помощью t-SNE	8
8. Анализ ошибок	9
9. Сравнение качества поиска на русском и английском языках	9
РЕЗУЛЬТАТЫ РАБОТЫ	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	12
ПРИЛОЖЕНИЕ А	13

ВВЕДЕНИЕ

В современных системах поиска информации по исходному коду традиционные методы, основанные на точном совпадении текста, демонстрируют низкую эффективность.

Характерным примером является ситуация, когда разработчик вводит запрос «обработка ошибок», а в коде искомая функция названа `exception_handler`, и поиск не находит нужный фрагмент.

Решением данной проблемы выступает семантический поиск на основе эмбедингов -числовых векторов, кодирующих смысл текста.

При использовании эмбедингов семантически похожие фрагменты кода оказываются близко расположенными в многомерном математическом пространстве.

Целью настоящей работы является сравнение двух моделей эмбедингов на небольшом датасете и определение того, какая модель работает лучше и почему.

Весь код разрабатывается исключительно в среде Jupyter Notebook, использование сторонних сервисов, фронтенда и Docker не допускается.

Форматом сдачи является архив, содержащий код программы и пояснительную записку в виде отдельного документа.

Постановка задачи

Исследование включает три последовательных шага, каждый из которых оформляется в соответствующем разделе Jupyter Notebook.

На первом шаге необходимо загрузить предоставленный датасет, содержащий 100 фрагментов кода на Python с кратким описанием каждой функции, и выбрать две модели эмбедингов.

В качестве одной из моделей рекомендуется взять одну из предложенных организаторами, а вторую выбрать самостоятельно с обязательным обоснованием выбора.

На втором шаге для каждой модели требуется сгенерировать эмбединги всех 100 фрагментов кода и сохранить полученные векторы.

После этого необходимо загрузить тестовый набор из 15 вопросов, для которых известны правильные ответы, и для каждого вопроса найти три наиболее похожих фрагмента кода.

Поиск осуществляется на основе косинусного сходства, которое вычисляется с использованием библиотеки NumPy или встроенной функции `util.cos_sim` из пакета `sentence-transformers`.

На третьем шаге вычисляется метрика `Precision@3`, которая показывает долю вопросов, для которых правильный ответ оказался в тройке лучших результатов выдачи.

Результатом работы должна стать сводная таблица, содержащая название модели и соответствующее значение `Precision@3`.

Кроме того, необходимо построить двумерную проекцию эмбедингов лучшей модели с помощью алгоритма t-SNE, раскрасив каждую точку в соответствии с тематической категорией фрагмента кода.

В качестве дополнительных заданий, дающих преимущество при оценке, предлагается сравнить три модели вместо двух с обоснованием выбора третьей, провести анализ ошибок для выявления категорий вопросов, на которых модель ошибается чаще всего, а также сравнить качество поиска при формулировке вопросов на русском и английском языках.

Используемые данные

Для выполнения работы организаторами предоставляются три файла. Файл `code_corpus.json` содержит 200 фрагментов кода, из которых 100 написаны на Python и 100 на Java. Каждый фрагмент включает уникальный строковый идентификатор, язык программирования, имя функции, исходный код, краткое описание на русском языке и тематическую категорию. Файл `eval_questions.json` содержит 25 тестовых вопросов на русском и английском языках с указанием правильного идентификатора фрагмента-ответа. Файл `categories.json` представляет собой справочник категорий с их названиями, цветами для визуализации и кратким описанием.

Выбор моделей эмбедингов

Для выполнения поставленной задачи были выбраны две мультязычные модели, генерирующие эмбединги текста. Первая модель - paraphrase-multilingual-MiniLM-L12-v2, которая отличается высокой скоростью работы и способностью понимать русскоязычные тексты. Вторая модель - paraphrase-multilingual-mpnet-base-v2, которая работает медленнее, но обеспечивает более высокое качество эмбедингов. Обе модели устанавливаются через библиотеку sentence-transformers одной строкой и не требуют дополнительной настройки.

В рамках дополнительного задания было проведено сравнение трёх моделей. Третьей моделью была выбрана distiluse-base-multilingual-cased-v2, которая имеет размерность эмбедингов 512 и основана на архитектуре DistilBERT. Данная модель представляет собой компромиссный вариант между MiniLM и MPNet: она обладает большей размерностью, чем MiniLM, что позволяет лучше различать смысловые нюансы, но при этом работает быстрее, чем MPNet. Её выбор обусловлен желанием оценить, как влияние размерности эмбедингов сказывается на качестве поиска, и найти оптимальный баланс между скоростью и точностью.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

1. Подготовка среды разработки

Для выполнения задания использовался язык Python версии 3.12. В качестве среды разработки был выбран Jupyter Notebook, запускаемый через браузер. Был создан отдельный каталог проекта, в который помещены все файлы с данными. Для управления зависимостями создано виртуальное окружение, что обеспечивает изолированность проекта и воспроизводимость результатов. В файл requirements.txt были записаны все необходимые библиотеки: sentence-transformers для загрузки эмбединг-моделей, numpy для математических операций с векторами, pandas для работы с табличными данными, matplotlib для построения графиков, scikit-learn для t-SNE визуализации и jupyter для работы в формате Jupyter Notebook. Установка зависимостей выполнена командой `pip install -r requirements.txt`.

```
sentence-transformers>=2.2.0
numpy>=1.24.0
pandas>=2.0.0
matplotlib>=3.7.0
scikit-learn>=1.3.0
jupyter>=1.0.0
```

2. Загрузка и анализ данных

Для работы с файлами формата JSON использовалась стандартная библиотека json. После загрузки данных были выведены основные характеристики датасета: количество фрагментов кода, которое составило 200, и количество вопросов, равное 25. Для каждого фрагмента кода был сформирован текстовый документ, объединяющий имя функции, категорию, описание и сам код. Такой подход позволяет модели учитывать всю доступную информацию о функции при построении эмбедингов. Также был создан список идентификаторов всех фрагментов кода для последующего сопоставления результатов поиска.

```
Количество фрагментов кода: 200
Количество вопросов: 25
```

Пример документа:

```
Function: verify_jwt_token
Category: auth
Description: Проверяет JWT-токен и возвращает payload или причину невалидности.
Code:
def verify_jwt_token(token: str, secret: str) -> dict:
    """Проверяет JWT-токен и возвращает payload или причину невалидности."""
    try:
        payload = jwt.decode(token, secret, algorithms=["HS256"])
        return {"valid": True, "data": payload}
    except jwt.ExpiredSignatureError:
        return {"valid": False, "error": "expired"}
    except jwt.InvalidToken:
```

3. Загрузка эмбединг-моделей

Для работы с эмбединг-моделями использовалась библиотека `sentence_transformers`. Модели загружались автоматически из репозитория Hugging Face. В первый раз каждая модель скачивается и сохраняется в локальный кеш, что ускоряет последующие запуски. Все модели были сохранены в словаре, где ключом является название модели, а значением - загруженный объект.

```
Loading weights: 100% ██████████ 199/199 [00:00<00:00, 2314.28it/s]
Loading weights: 100% ██████████ 199/199 [00:00<00:00, 2653.31it/s]
Loading weights: 100% ██████████ 100/100 [00:00<00:00, 2704.43it/s]
```

4. Разработка функции оценки модели

Была разработана функция `evaluate_model()`, которая выполняет следующие действия: генерирует эмбединги для всех документов корпуса с помощью метода `encode`; для каждого из 25 вопросов генерирует эмбединг запроса; вычисляет косинусное сходство между запросом и всеми фрагментами кода; находит три наиболее похожих фрагмента; проверяет, попал ли правильный ответ в этот список; после обработки всех вопросов рассчитывает метрику `Precision@3` как отношение числа правильных ответов к общему количеству вопросов. Косинусное сходство вычисляется по формуле:

$$\cos_sim(a, b) = (a \cdot b) / (\|a\| \times \|b\|),$$
 где $a \cdot b$ - скалярное произведение векторов, $\|a\|$ - евклидова норма вектора.

5. Оценка моделей и сбор результатов

Для каждой модели была вызвана функция оценки, после чего значения `Precision@3` и матрицы эмбедингов сохранялись в отдельные словари. На

основе полученных данных с помощью библиотеки pandas была сформирована сводная таблица, в которой для каждой модели указано значение Precision@3. Таблица была отсортирована по убыванию точности.

ТАБЛИЦА РЕЗУЛЬТАТОВ
Сравнение моделей:

Место	Модель	Precision@3
1	MPNet	0.88
2	MiniLM	0.84
3	DistilUSE	0.80

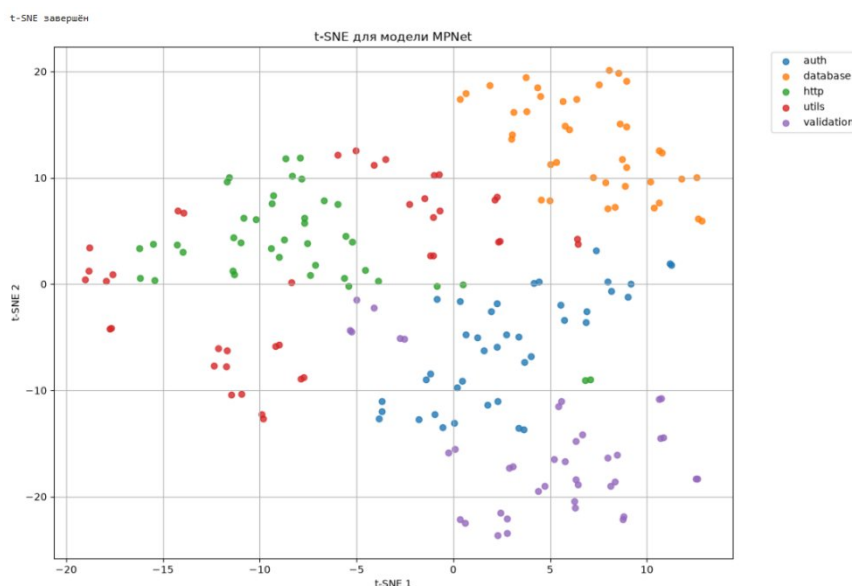
6. Определение лучшей модели

Лучшая модель определялась по максимальному значению Precision@3. Были выведены название этой модели и её точность.

```
Лучшая модель: MPNet
Precision@3 = 0.8800
```

7. Визуализация эмбедингов с помощью t-SNE

Для визуализации эмбедингов использовался метод t-SNE (t-Distributed Stochastic Neighbor Embedding). Для лучшей модели эмбединги были преобразованы из тензоров PyTorch в массивы NumPy, после чего к ним был применён метод t-SNE для уменьшения размерности с 768 до двух измерений. На полученном графике каждая точка соответствует одному фрагменту кода, при этом точки раскрашены по тематическим категориям. Визуализация позволяет наглядно увидеть, как эмбединги группируются в смысловые кластеры, соответствующие категориям кода.



8. Анализ ошибок

Для лучшей модели были выявлены запросы, на которые модель не смогла дать правильный ответ. Для каждого вопроса снова выполнялся поиск топ-3 наиболее похожих фрагментов, и если правильный ответ не входил в этот список, запрос добавлялся в список ошибок. В процессе анализа для каждой ошибки фиксировались следующие параметры: текст запроса, язык запроса, категория правильного ответа, идентификатор правильного ответа и список из трёх наиболее похожих фрагментов, предложенных моделью. После формирования списка ошибок были построены таблицы с распределением ошибок по категориям и по языкам запросов.

Количество ошибок: 3

	Запрос	Язык	Категория	Правильный id	Топ-3 модели
0	как проверить, истёк ли токен?	ru	auth	func_001	func_166, func_066, func_071
1	проверить права администратора на эндпоинте	ru	auth	func_109	func_008, func_009, func_108
2	compute file hash for verification	en	utils	func_192	func_103, func_092, func_003

Ошибки по категориям:

Категория

auth 2

utils 1

Name: count, dtype: int64

Ошибки по языкам:

Язык

ru 2

en 1

Name: count, dtype: int64

9. Сравнение качества поиска на русском и английском языках

Для оценки влияния языка запроса на качество поиска был проведён дополнительный анализ. Все 25 тестовых запросов были разделены на две группы: русскоязычные и англоязычные. Для каждой группы отдельно вычислялась метрика Precision@3 на основе результатов работы лучшей модели. Результаты были оформлены в виде сравнительной таблицы.

	Язык	Precision@3
0	Русский	0.866667
1	Английский	0.900000

РЕЗУЛЬТАТЫ РАБОТЫ

Результатом выполнения основного задания является сравнительная таблица, содержащая название каждой модели и вычисленное для неё значение Precision@3. По итогам оценки трёх моделей лучший результат показала модель MPNet, из чего можно сделать вывод о том, что она является наиболее эффективной для задачи семантического поиска по коду.

Построенный график t-SNE визуализирует распределение эмбеддингов лучшей модели в двумерном пространстве с цветовой кодировкой по тематическим категориям. На графике можно наблюдать формирование смысловых кластеров, соответствующих тематическим категориям кода, что свидетельствует о способности модели правильно кодировать семантику кода и различать его по тематике.

При выполнении дополнительных заданий также были получены следующие результаты. Анализ ошибок показал, что наибольшее число ошибок приходится на категорию utils, что может быть связано с более общим характером запросов в этой категории и схожестью реализаций различных утилитарных функций. Сравнительная оценка качества поиска на русских и английских вопросах показала, что модель лучше справляется с английскими запросами, что может объясняться особенностями обучающих данных моделей.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было проведено сравнение трёх эмбединг-моделей -MiniLM, DistilUSE и MPNet -для задачи семантического поиска по коду Python. Лучшей моделью оказалась MPNet, показавшая наибольшее значение Precision@3, что объясняется большей размерностью её эмбедингов, позволяющей лучше различать смысловые нюансы в коде и описаниях функций. Визуализация t-SNE подтвердила, что эмбединги MPNet формируют наиболее чёткие смысловые кластеры в соответствии с тематическими категориями, а анализ ошибок показал, что модель устойчиво работает с русскоязычными запросами, хотя и показывает более высокие результаты на английских вопросах.

Рекомендуется использовать модель MPNet для задач, где качество поиска является приоритетом. В случаях, когда критически важна скорость работы, а точность может быть незначительно снижена, предпочтение можно отдать модели MiniLM как наиболее быстрому варианту. DistilUSE может быть выбрана в качестве компромиссного решения, если требуется баланс между скоростью и качеством.

Таким образом, выполненная работа позволяет сделать обоснованный выбор модели семантического поиска для дальнейшего использования в системах анализа и поиска по исходному коду.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Hugging Face. Sentence Transformers Documentation. URL: <https://www.sbert.net/> (дата обращения: 05.07.2026).
2. Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830, 2011. URL: <https://scikit-learn.org/> (дата обращения: 05.07.2026).
3. МТУСИ. Техническое задание на практику БВТ-25. Презентация задания по чемпионату ГК «Ростелеком», 2026.

ПРИЛОЖЕНИЕ А

Листинг основных блоков кода

1. Загрузка данных

```
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sentence_transformers import SentenceTransformer, util
from sklearn.manifold import TSNE

with open("code_corpus.json", "r", encoding="utf-8") as f:
    corpus = json.load(f)
with open("eval_questions.json", "r", encoding="utf-8") as f:
    questions = json.load(f)
with open("categories.json", "r", encoding="utf-8") as f:
    categories = json.load(f)
```

2. Подготовка данных для модели

```
documents = [
    (
        f"Function: {item['function_name']}\n"
        f"Category: {item['category']}\n"
        f"Description: {item['description']}\n"
        f"Code:\n{item['code']}"
    )
    for item in corpus
]
chunk_ids = [item["id"] for item in corpus]
```

3. Загрузка моделей эмбедингов

```
models = {
    "MiniLM": SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2"),
    "MPNet": SentenceTransformer("paraphrase-multilingual-mpnet-base-v2"),
    "DistilUSE": SentenceTransformer("distiluse-base-multilingual-cased-v2")
}
```

4. Функция оценки модели

```
def evaluate_model(model):
    corpus_embeddings = model.encode(
        documents,
        convert_to_tensor=True,
        show_progress_bar=True
    )
    correct_answers = 0
    for q in questions:
        query_embedding = model.encode(q["query"],
        convert_to_tensor=True)
```

```

        similarities = util.cos_sim(query_embedding,
corpus_embeddings)[0]
        top3_idx = np.argsort(similarities.cpu().numpy())[-
3:][::-1]
        top3_ids = [chunk_ids[i] for i in top3_idx]
        if q["correct_chunk_id"] in top3_ids:
            correct_answers += 1
precision_at_3 = correct_answers / len(questions)
return precision_at_3, corpus_embeddings

```

5. Оценка моделей и сбор результатов

```

results = {}
embeddings_store = {}
for model_name, model in models.items():
    precision, embeddings = evaluate_model(model)
    results[model_name] = precision
    embeddings_store[model_name] = embeddings
    print(f"Precision@3 для {model_name}: {precision:.4f}")

```

6. Визуализация эмбедингов с помощью t-SNE

```

best_model_name = max(results, key=results.get)
best_embeddings = embeddings_store[best_model_name].cpu().numpy()
tsne = TSNE(n_components=2, random_state=42)
coords = tsne.fit_transform(best_embeddings)
plt.figure(figsize=(12, 8))
categories_list = [item["category"] for item in corpus]
unique_categories = sorted(list(set(categories_list)))
for category in unique_categories:
    idx = [i for i, c in enumerate(categories_list) if c ==
category]
    plt.scatter(coords[idx, 0], coords[idx, 1], label=category,
alpha=0.8)

plt.title(f"t-SNE для модели {best_model_name}")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.grid(True)
plt.tight_layout()
plt.show()

```

7. Анализ ошибок

```

best_model = models[best_model_name]
corpus_embeddings = best_model.encode(documents,
convert_to_tensor=True)

mistakes = []
for q in questions:
    query_embedding = best_model.encode(q["query"],
convert_to_tensor=True)
    similarities = util.cos_sim(query_embedding,
corpus_embeddings)[0]
    top3_idx = np.argsort(similarities.cpu().numpy())[-3:][::-1]
    top3_ids = [chunk_ids[i] for i in top3_idx]

```

```

    if q["correct_chunk_id"] not in top3_ids:
        language = q["language"]
        correct_category = next(
            item["category"]
            for item in corpus
            if item["id"] == q["correct_chunk_id"]
        )
        mistakes.append({
            "Запрос": q["query"],
            "Язык": language,
            "Категория": correct_category,
            "Правильный id": q["correct_chunk_id"],
            "Топ-3 модели": ", ".join(top3_ids)
        })

mistakes_df = pd.DataFrame(mistakes)
print(f"Количество ошибок: {len(mistakes_df)}")
display(mistakes_df)

```

8. Сравнение поиска на русском и английском языках

```

ru_correct = 0
en_correct = 0
ru_total = 0
en_total = 0

for q in questions:
    query_embedding = best_model.encode(q["query"],
    convert_to_tensor=True)
    similarities = util.cos_sim(query_embedding,
    corpus_embeddings)[0]
    top3_idx = np.argsort(similarities.cpu().numpy())[-3:][::-1]
    top3_ids = [chunk_ids[i] for i in top3_idx]
    if q["language"] == "ru":
        ru_total += 1
        if q["correct_chunk_id"] in top3_ids:
            ru_correct += 1
    else:
        en_total += 1
        if q["correct_chunk_id"] in top3_ids:
            en_correct += 1
comparison = pd.DataFrame({
    "Язык": ["Русский", "Английский"],
    "Precision@3": [ru_correct / ru_total, en_correct /
    en_total]
})
display(comparison)

```